

1 Morale

Si presenta l'implementazione di un'applicazione distribuita che realizza il metodo di gestione del lavoro basato su **kanban**. I principi che si sono voluti seguire in fase di sviluppo sono:

- Protocollo di trasmissione livello *application* basato su stringhe ASCII separate da ritorni carrello (`\n`), leggibili da uomo, ispirato a SMTP. Queste stringhe vanno a rappresentare i *comandi* dell'applicazione in maniera espressiva e su cui è facile fare debugging;
- Comandi fra server e client inviati su TCP, comandi fra client inviati su UDP (con minima affidabilità basata su ritrasmissioni). L'uso di socket TCP e UDP sulla stessa porta permette di discriminare gli utenti per porte anche nelle connessioni peer-to-peer;
- Design *modulare*, dove si divide la funzionalità in appositi moduli *core*, *net*, ecc... lato client e server, per maggiore leggibilità;
- Approccio il più semplice possibile: no allocazioni sull'heap, client e server ciascuno su un unico thread di esecuzione. Questo è stato fatto per aumentare la robustezza del codice e ridurre la possibilità di errori, con il contro di minore scalabilità.

2 Costanti e strutture condivise

Esistono alcune strutture condivise fra client e server: queste vengono definite una sola volta per evitare di riprodurre codice.

- `card/`: definisce il tipo `card` di una kanban:

```
1 typedef struct {
2     card_id id; // id (se e' 0 la card e' nulla)
3     char desc[CARD_DESC_LEN]; // testo attivita'
4     int user; // utente che la sta implementando o l'ha implementata
5     struct tm timestamp; // timestamp dell'ultima modifica
6 } card;
```

Le card gestite dal server vengono allocate in una pool statica definita in `server/pool.h`: si possono ottenere e liberare nuove card con le funzioni `alloc_card()` e `free_card(card *p)`, dalla semantica simile alle `malloc()` e `free()` della `stdlib`. Questo permette di evitare allocazioni sull'heap, riducendo la complessità e il rischio di perdite di memoria. Puntatori a card vengono organizzati in un'apposita matrice che rappresenta le colonne della lavagna: `card *columns[NUM_COLS][MAX_CARDS_PER_COL]` in `server/core.h`.

- `cmd/`: definisce il tipo `cmd` per i comandi scambiati fra client e server, e fra più client in modalità peer-to-peer:

```
1 typedef struct {
2     cmd_type type; // tipo di comando
3     const char *args[MAX_CMD_ARGS]; // argomenti del comando, in una lista terminata da NULL
4 } cmd;
```

Per comodità si è scelto di rappresentare gli argomenti come una lista di puntatori a stringhe, terminati da `NULL`. Il formato dei comandi in transito sulla rete è generalmente il tipo di comando in maiuscolo. Fanno eccezione i comandi con argomenti:

Comando	Formato
CREATE_CARD	CREATE_CARD <id> <colonna> <descrizione>
ACK_CARD	ACK_CARD <id>
MOVE_CARD	MOVE_CARD <id> <colonna>
HANDLE_CARD	HANDLE_CARD <id> <descrizione>
SEND_USER_LIST	SEND_USER_LIST <porta> ...
ACK_REVIEW_CARD	ACK_REVIEW_CARD <voto>

I comandi OK, ERR e ACK_REVIEW_CARD vengono forniti come ack. A ogni comando può seguire un commento.

- `core_const.h` e `net_const.h` contengono costanti relative rispettivamente ai *core* (numero massimo di client, ecc..) e ai moduli di rete (indirizzi, numeri di porta, ecc...).

3 Gestione rete

L'idea dietro i moduli di rete è che questi separano la logica di lettura e scrittura su socket dai moduli *core*, che devono preoccuparsi solo della logica relativa alla kanban. In particolare, abbiamo che:

- Le operazioni di *ricezione* (`recv()`) avvengono in multiplexing, e quando possibile. Ciò significa appena si ha tempo per il server, e quando si hanno "*momenti morti*" per il client: e.g. quando si fanno richieste o quando si va in attesa simulando l'elaborazione delle card. Il contro di questo approccio è che si possono avere lievi imprecisioni temporali per la gestione dei timer del server;
- Le operazioni di *invio* (`send()`) sono sincrone e bloccanti, in quanto la mole di comandi inviati è ridotta.

Il compito vero e proprio dei moduli di rete è quindi schematizzabile come segue:

1. Ottenere linee separate da `\n` dallo stack di rete del sistema operativo e deserializzarle in comandi;
2. Smistare tali comandi fra i vari moduli. Questo si fa sfruttando una mappa definita sui tipi comando:

```
1 typedef enum {
2     CORE, // comandi standard
3     WATCH, // comandi di controllo client (PING_USER, PONG_LAVAGNA)
4     PEER, // comandi di revisione fra peer (REVIEW_CARD, ACK_REVIEW_CARD)
5 } mod_type;
```

3. Inviare indietro risposte, serializzate da comandi, ottenuti sempre dai moduli.

4 Server

Per il server questa corrispondenza è diretta, in quanto tutti i comandi ricevuti sono provenienti da client, e il modulo di rete deve quindi limitarsi a smistarli fra modulo *core* e modulo *watch*. Non esiste quindi una funzione `recv()`, ma una `listen_net()` che gestisce in multiplexing le richieste arrivate al server, oltre che ai timer usati per gestire il modulo *watch*. Questa si basa sulla `select()`:

```
1 void listen_net() {
2     // gestisci i timer
3     for (int i = 0; i < MAX_CLIENTS; i++)
4         handle_timer(&connections[i]); // sostanzialmente un tick del timer
5
6     // scansiona con la select
7     select(fdmax + 1, &read_set, NULL, NULL, &tv);
8
9     for (int i = 0; i <= fdmax; i++) {
10        // controlla che si qualcosa da leggere
11        if (!FD_ISSET(i, &read_set))
12            continue;
13
14        if (i == listen_sock) { // accetta nuovo client
15            accept_client();
16        } else if (i == STDIN_FILENO) { // gestisci console amministratore
17            handle_client(&admin_conn);
18        } else { // gestisci client
19            handle_client_sock(i); // chiama handle_client()
```

```

20     }
21 }
22 }

```

Notiamo che si prevede un client speciale, con socket `STDIN_FILENO`, inteso come l'“amministratore”. Ne si prende il controllo interagendo con la riga di comando offerta dal server stesso, e viene fornito per spostare card e crearne di nuove. Si rende poi disponibile una `send_client(client_id cl_id, const cmd *cm)`. La gestione dello stato del sistema e delle card è delegata al modulo `core`, che deve solo portare avanti l'operazione richiesta sulla base del tipo di `cmd` fornito in `handle_command(client_id cl_id, const cmd *cm)` (chiamata da `handle_client()`). Notiamo poi che i moduli `net` e `core` del server usano astrazioni diverse per i client, in modo da mantenere in ogni modulo solo l'informazione necessaria. Un client per il modulo rete è rappresentato da una *connessione*:

```

1 typedef struct {
2     int sock; // socket della connessione (se e' 0 la connessione e' nulla)
3     unsigned short port; // porta della connessione
4     char read_buf[CMD_BUF_SIZE]; // buffer di lettura
5     int read_len; // caratteri letti in buffer
6 } connection;

```

mentre nel modulo `core` si mantiene informazione relativa allo stato dell'utente, alle card fornite, e di utilità al modulo `watch`:

```

1 typedef struct {
2     client_id id; // id (se e' 0 il client e' nullo)
3     client_sts sts; // stato
4     card *handling; // puntatore alla card che sta gestendo (se e' NULL sta aspettando una card)
5     struct timespec deadline; // deadline del timer
6     int sent_pong; // un flag che rappresenta se si sta aspettando un ping dal client
7 } client;

```

Si nota la stretta dipendenza fra il modulo `watch` e il modulo `core`: per facilitare la loro interoperazione quest'ultimo fornisce appositamente due header, `core.h` e `core_watch.h`, di cui il secondo più intimo e riservato al modulo `watch`.

5 Client

Per il client si ha una complicazione data dal fatto che è il modulo `core` a portare avanti l'elaborazione, effettuando operazioni di ricezione e invio su propria discrezione. Allo stesso tempo, il modulo di rete deve essere pronto a gestire comandi provenienti sia dal server che da altri client, e questi comandi potrebbero dover essere gestiti da uno qualsiasi fra i moduli `core`, `watch` e `rev`. Per questo si rende disponibile una funzione `recv_multi(cmd *cm, int block)`, che gestisce in multiplexing le richieste, e smista finché non si ottiene un comando diretto al modulo `core`. In tal caso si restituisce il comando, rimandando alla prossima chiamata la gestione delle richieste successive. Il flag `block` permette inoltre di specificare se si vuole che la funzione blocchi fino alla ricezione di un comando diretto a `core`, o se si vuole uscire dopo un timeout di 1 secondo (usato per l'implementazione dell'attesa casuale). Con questo approccio si riescono a gestire le richieste dirette al client nei “*momenti morti*” accennati sopra:

```

1 int recv_multi(cmd *cm, int block) {
2     while (1) {
3         // scansiona con la select
4         if(select(fdmax + 1, &read_set, NULL, NULL, block ? NULL : &tv) == 0) return 0;
5
6         // c'e' qualcosa sul socket UDP?
7         unsigned short who;
8         if (FD_ISSET(udp_sock, &read_set))
9             recv_peer(&who, cm);
10            if (handle_cmd(who, cm) > 0)
11                return 1; // e' core, esci
12
13            // c'e' qualcosa sul socket TCP?
14            if (FD_ISSET(tcp_sock, &read_set))
15                recv_server(cm);
16            if (handle_cmd(0, cm) > 0)
17                return 1; // e' core, esci
18    }
19 }

```

Si rendono poi disponibili una `send_server(const cmd *cm)` per l'invio a server e una `send_peer(unsigned short who, const cmd *cm)` per l'invio a peer. L'implementazione del client diventa quindi semplice: basta chiamare la `recv_multi()` per il prossimo comando `core`, con gli altri comandi gestiti dietro le quinte. Vediamo a scopo di esempio come ci si registra al server:

```

1 int hello() {
2     // richiedi registrazione
3     cmd cm = {.type = HELLO, .args = {"richiedo la mia registrazione"}};
4     send_server(&cm);
5
6     // attendi ack
7     cmd rep = {0};
8     int ret = recv_multi(&rep, BLOCK);
9     return rep.type == OK;
10 }

```

Funzioni di questo tipo (definite nel modulo `core` del client) vengono chiamate in un loop che ottiene card, aspetta un tempo casuale, richiede i peer e li interroga per le revisioni, e restituisce la conferma di aver processato la card al server.

6 Compilazione

Si rende disponibile un Makefile per la compilazione con `make`. `make` viene usato anche per avviare dei test:

```

1 # compila la lavagna
2 make lavagna
3
4 # esegue la lavagna con un set di card predefinito, specificato nel file dat/cards.txt
5 make run_lavagna
6
7 # compila il client
8 make client
9
10 # esegue 4 client
11 make run_clients
12
13 # esegue un client su una certa porta
14 make run_client ARGS="<porta>"
15
16 # ripulisce gli oggetti
17 make clean

```